Quantum Phase Estimation

1 Understanding the Need

Quantum phase estimation is a quantum algorithm to estimate the phase corresponding to an eigenvalue of a given unitary operator. Because the eigenvalues of a unitary operator always have unit modulus, they are characterized by their phase, and therefore, the algorithm can be equivalently described as retrieving either the phase or the eigenvalue itself.

It has various use cases, from cracking passwords to building artificial intelligence.

2 Building an Intuition for the Algorithm

Say you have a vector in some space. There is a function that rotates this vector by some angle, called phase, each time it is applied. You want to find out what this function is, and effectively, this angle phase. How would you do this?

Well, we might not know what our vector looks like after each time the function rotates it, but we do know what it looks like after being rotated by 90°, 180°, or 360°. So, keep applying the function until the vector rotates by 90°. If it took n rotations to reach 90°, our phase is simply $90°/n$

This is the essence of QPE.

Applying this analogy classically takes exponential time. Quantum phase estimation, however, allows us to solve this problem in polynomial time.

3 Looking at the Algorithm Formulation

If you are reading this, I assume you are familiar with standard quantum gates and quantum circuit diagrams. Given below is what a 3-qubit Quantum Phase Estimation implementation for the T-Gate looks like.

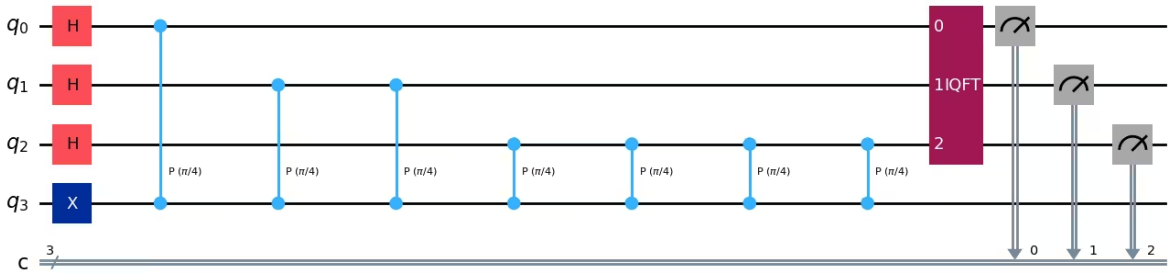

Figure 1: A 3-qubit Quantum Phase Estimation circuit for the T-Gate.

3.1 The Circuit Setup

QPE uses two registers:

- **Counting register** (top qubits, e.g., 3 qubits in the example), these start in $|0\rangle$ and will hold the answer (the phase) at the end.
- **Eigenstate register** (bottom qubits), prepared in the eigenstate $|\psi\rangle$ of our unitary U. This is the "vector" from our analogy.

More counting qubits means more precision in your phase estimate, just like using more decimal digits gives a more precise number.

3.2 Step-by-Step Understanding of the Algorithm

Step 1. Superposition. Apply a Hadamard gate to every qubit in the counting register. This puts each one into an equal superposition of $|0\rangle$ and $|1\rangle$. Instead of testing rotations one at a time (the classical, exponential way), superposition lets you test all of them at once.

Step 2. Controlled rotations. Apply controlled-U operations from each counting qubit onto the eigenstate register, doubling the number of applications each time:

- 1st counting qubit $\rightarrow$ controlled-U^1
- 2nd counting qubit $\rightarrow$ controlled-U^2
- 3rd counting qubit $\rightarrow$ controlled-U^4

Each controlled-U "kicks back" a piece of the phase information into the counting register (this is called phase kickback). This doubling trick is what allows QPE to extract the phase in polynomial time rather than checking rotations one by one.

Step 3. Inverse Quantum Fourier Transform (QFT†). The counting register now holds the phase information, but scrambled across superposition. The inverse QFT converts this into a clean binary number upon measurement, similar to how a Fourier transform converts a repeating waveform into a frequency spectrum.

It's alright if you don't understand what a Fourier transform is exactly yet. Here, it simply converts the quantum signal into something measurable.

Step 4. Measurement. Measure the counting qubits. The result is a binary string that, when divided by 2^n (n = number of counting qubits), gives you the estimated phase.

4 Further Reading

- Quantum Phase Estimation Algorithm, IBM Quantum Learning: IBM Quantum Learning